Tutorato 4

Matteo Mazzaretto

10 Novembre 2026



L'esercizio: dList<T>

Lista doppiamente concatenata con:

- gestione della memoria **senza condivisione** (big 3)
- costruttore `dList(unsigned int k, const T& t)`
- `insertFront` e `insertBack` in **O(1)**
- `operator<` con ordinamento **lessicografico**
- `const_iterator` (con l'iteratore past-the-end)

Oggi: **due soluzioni** a confronto, quali idee cambiano e perché.

Cosa hanno in comune

UNIVERSITÀ DI PADOVA

Struttura

- classe annidata privata `nodo` con `info`, `prev`, `next`
- puntatori `first` e `last`
- lista vuota $\iff$ `first == last == nullptr`

Inserimento in coda, O(1)

```
void insertBack(const T& t) {
  nodo* n = new nodo(t, last, nullptr);
  if(last) last->next = n;
  else first = n;
  last = n;
}
```

Avere `last` è ciò che rende O(1) anche `insertBack`.

Confronto: le cinque idee

1. Come rappresentare il **past-the-end**
2. I **casi limite** dell'iteratore
3. **Ricorsione** o **cicli** nelle funzioni di appoggio
4. Cosa richiediamo al tipo `T` (==? costruttore di default?)
5. L'ordine delle operazioni in `operator=`

Soluzione A (riferimento) e **Soluzione B** (variante).

Idea 1: il past-the-end

Soluzione A: puntatore "oltre" l'ultimo

```
end() {
  return const_iterator(last+1, true);
}
```

`++` sull'ultimo nodo fa `++ptr`, `--` da end fa `--ptr`.

Soluzione B: flag sul nodo finale

```
end() {
  return const_iterator(last, true);
}
```

`++` sull'ultimo nodo accende `past_the_end`, `--` da end lo spegne.

Idea 1: i dettagli

A

```
const_iterator& operator++() {
  if(ptr && !pte) {
    if(ptr->next == nullptr)
      {++ptr; pte = true;}
    else ptr = ptr->next;
  }
  return *this;
}
```

B

```
const_iterator& operator++() {
  if(iter && !past_the_end) {
    if(iter->next) iter = iter->next;
    else past_the_end = true;
  }
  return *this;
}
```

- A ha un puntatore a un singolo oggetto che può avanzare di uno (one-past-the-end) e poi tornare indietro, ma `ptr` non punta più a un nodo, e bisogna ricordarsi che `pte` lo protegge
- B non fa aritmetica sui puntatori: `iter` punta sempre a un nodo vero

Idea 2: il codice di --, begin() e end()

UNIVERSITÀ DI PADOVA

A

```
const_iterator& operator--() {
  if(ptr) {
    if(pte) {--ptr; pte = false;}
    else ptr = ptr->prev;
  }
  return *this;
}

const_iterator begin() const {
  if(!first) return const_iterator();
  return const_iterator(first);
}
const_iterator end() const {
  if(!first) return const_iterator();
  return const_iterator(last+1, true);
}
```

B

```
const_iterator& operator--() {
  if(iter) {
    if(past_the_end) past_the_end = false;
    else if(iter->prec) iter = iter->prec;
  }
  return *this;
}

const_iterator begin() const {
  if(!first) return const_iterator();
  return const_iterator(first);
}
const_iterator end() const {
  if(!first) return const_iterator();
  return const_iterator(last,true);
}
```

- A: su `begin()`, `ptr = ptr->prev` porta a `nullptr` e l'iteratore diventa "indefinito"
- B: l'`else if(iter->prec)` fa restare l'iteratore su `begin()`

Idea 2: il codice di * e ->

A: segnala l'errore

```
const T& operator*() const {
  if(pte) {
    const T* p = nullptr;
    return *p;
  }
  return ptr->info;
}

const T* operator->() const {
  if(pte) return nullptr;
  return &(ptr->info);
}
```

B: nessun controllo

```
const T& operator*() const {
  return iter->info;
}

const T* operator->() const {
  return &(iter->info);
}

bool is_valid() const {
  return iter != nullptr
      && !past_the_end;
}
```

- A: `*end()` fallisce in modo evidente, `end()->` dà `nullptr`
- B: `*end()` restituisce l'ultimo elemento senza avvisare, quindi il controllo spetta a chi chiama (`is_valid()`)

operator->: come funziona

Con un puntatore

```
Punto* p = ...;
p->x;       // uguale a (*p).x
```

Vogliamo lo stesso con l'iteratore

```
auto it = l.begin();
(*it).x;    // operator*
it->x;      // operator->
```

Cosa fa il compilatore

```
it->x
// diventa
(it.operator->())->x
```

Quindi la funzione restituisce un puntatore

```
const T* operator->() const {
  return &(iter->info);
}
```

`operator->` non accede al campo: restituisce un **puntatore all'elemento**, e il compilatore ci riapplica `->x`.
Non sarà utilizzato per tutti i tipi T, ad esempio non si può utilizzare sul tipo **int**.

Idea 2: i casi limite

Situazione	A	B
`--begin()`	`ptr` diventa `nullptr`: iteratore “indefinito”	resta su `begin()`
lista vuota	`begin()` e `end()`: `(nullptr, false)`	`begin()` e `end()`: `(nullptr, true)`
`*end()`	si dereferenzia `nullptr`	restituisce l’ultimo elemento
`end()->`	restituisce `nullptr`	punta all’ultimo elemento

In entrambe dereferenziare `end()` è un errore. In A si nota subito (crash), in B passa in silenzio: per questo B ha `is_valid()` per controllare prima di dereferenziare.

Idea 3: ricorsione o cicli

UNIVERSITÀ DI PADOVA

A: ricorsiva

```
static void distruggi(nodo* p) {
  if(p) {
    distruggi(p->next);
    delete p;
  }
}
```

B: iterativa

```
static void distruggi(nodo* p) {
  while(p) {
    nodo* s = p->next;
    delete p;
    p = s;
  }
}
```

- la ricorsione usa **stack proporzionale alla lunghezza** della lista: con liste molto lunghe può esaurirlo
- la versione ricorsiva è però più breve e vicina alla definizione induttiva

Idea 3: ricorsione o cicli

A: ricorsiva

```
static void copia(nodo* primo, nodo*& f, nodo*& l) {
  if(primo == nullptr) {
    f=l=nullptr;
    return;
  }
  copia(primo->next,f,l);
  f = new nodo(primo->info,nullptr,f);
  if(f->next == nullptr)
    l = f;
  else
    (f->next)->prev = f;
}
```

B: iterativa

```
static void copia(nodo* orig, nodo*& f, nodo*& l){
  f = l = nullptr;
  for(; orig; orig = orig->next){
      nodo* n = new nodo(orig->info, nullptr, l);
      if(l) l->next = n;
      else  f = n;
      l = n;
  }
}
```

Idea 3: ricorsione o cicli

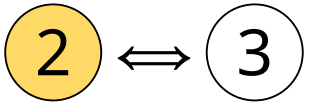
Ricorsiva: scende fino in fondo, poi costruisce **risalendo**

`copia(1,2,3)` ⟶ `copia(2,3)` ⟶ `copia(3)` ⟶ `copia(vuota)`

Cosa restituisce ogni chiamata, dal basso verso l'alto:

`copia(vuota)` *lista vuota*

`copia(3)` 

`copia(2,3)` 

`copia(1,2,3)` 

I nodi nascono dall'**ultimo al primo**: "copia il resto, poi aggiungi me davanti".

Idea 3: ricorsione o cicli

Iterativa: scorre l'originale e aggiunge **in coda**

`orig` legge: 1 ⟶ 2 ⟶ 3

La nuova lista dopo ogni giro del ciclo:

`inizio` *lista vuota* (`f = l = nullptr`)

`giro 1` (1) (`f = l = 1`)

`giro 2` (1) ⟺ (2) (`l = 2`)

`giro 3` (1) ⟺ (2) ⟺ (3) (`l = 3`)

I nodi nascono dal **primo all'ultimo**: "per ogni nodo, aggiungimi in coda".

Idea 4: il confronto lessicografico

A: <, < invertito, poi ==

```
if(f1->info < f2->info) return true;
if(f2->info < f1->info) return false;
if(f1->info == f2->info)
  return rec_lex_ord(f1->next,
                     f2->next);
return false;
```

B: solo <

```
while(a && b) {
  if(a->info < b->info) return true;
  if(b->info < a->info) return false;
  a = a->next; b = b->next;
}
return !a && b;
```

Idea 4 bis: requisiti su T

A: un solo costruttore con valori di default

```
dList(unsigned int k = 0, const T& t = T());
nodo(const T& t = T(), nodo* p = 0, nodo* n = 0);
```

- comodo: copre costruttore a 0, 1 e 2 parametri
- ma `T` deve avere il **costruttore di default**, anche se `k` è dato

B: costruttore di default e `dList(k, t)` separati, `nodo` senza default per `T`

- funziona anche con tipi senza costruttore di default

Idea 5: operator=

A: copia, poi distrugge

```
if(this != &d) {
  nodo *f, *l;
  copia(d.first, f, l);
  distruggi(first);
  first = f; last = l;
}
```

B: distrugge, poi copia

```
if(this != &d) {
  distruggi(first);
  copia(d.first, first, last);
}
```

- se new fallisce durante la copia, in A l'oggetto di partenza resta **intatto**
- in B la vecchia lista è già persa
- qui **A è la scelta migliore**

Dove A è meglio, dove B

A

- `operator=` robusto
- codice più breve, vicino alla definizione ricorsiva
- segnala esplicitamente `*end()`

B

- niente `last+1`: `iter` punta sempre a un nodo
- cicli: nessun rischio di stack
- meno requisiti su `T` (==, costruttore di default)

Cosa portare via

- un iteratore past-the-end si può rappresentare in più modi: conta **dichiarare l'invariante** (cosa significa `ptr`, cosa significa `pte`)
- definire i **casi limite** (`--begin()`, `*end()`, lista vuota) prima di scrivere il codice
- la ricorsione è elegante, ma ha un costo in stack
- chiedere a `T` il **minimo** indispensabile
- in `operator=`: **prima si copia, poi si distrugge**

Il nuovo main

UNIVERSITÀ DI PADOVA

```cpp
struct Punto { int x, y; };   // da mettere prima del main

int main(){
  dList<int>::const_iterator it = x.begin();
  std::cout << "primo di x: " << *it << std::endl;
  ++it;
  std::cout << "secondo di x + 1: " << *it + 1 << std::endl;

  const int& r = *it;                       // riferimento all'elemento, nessuna copia
  std::cout << "r = " << r << std::endl;

  int somma = 0;
  for(dList<int>::const_iterator cit = y.begin(); cit != y.end(); ++cit)
    somma += *cit;
  std::cout << "somma di y = " << somma << std::endl;

  dList<int>::const_iterator ult = y.end();
  --ult;                                    // da end() si torna sull'ultimo elemento
  std::cout << "ultimo di y: " << *ult << std::endl;

  // *it = 5;                               // ERRORE: const_iterator, l'elemento è const
```

Il nuovo main

UNIVERSITÀ DI PADOVA

```
// ---------- uso di -> (accesso a un membro dell'elemento) ----------
  dList<std::string> s;
  s.insertBack("campana"); s.insertBack("cavolo"); s.insertBack("ok");
  for(dList<std::string>::const_iterator cit = s.begin(); cit != s.end(); ++cit)
    std::cout << *cit << " ha " << cit->size() << " lettere" << std::endl;

  Punto a = {1, 2}, b = {3, 4};
  dList<Punto> p;
  p.insertBack(a); p.insertBack(b);
  for(dList<Punto>::const_iterator cit = p.begin(); cit != p.end(); ++cit)
    std::cout << "(" << cit->x << ", " << cit->y << ") ";
  std::cout << std::endl;

  dList<Punto>::const_iterator pit = p.begin();
  std::cout << pit->x << " " << (*pit).x << std::endl;   // sono la stessa cosa

  // pit->x = 9;                          // ERRORE: const_iterator
  // x.begin()->size();                   // ERRORE: int non ha membri, qui serve *
}
```

ATTENZIONE, È NEL CODICE DEL PROF